SSD1306 OLED Controller, GDDRAM Architecture & Pure-C Driver

1. Overview of the SSD1306 OLED Controller

The **SSD1306** is a single-chip CMOS OLED/PLED driver with controller for organic / polymer light-emitting diode dot-matrix graphic display systems. In this project, a **0.96-inch 128x64 Monochrome OLED module** is used. Unlike LCD panels that require an external CCFL or LED backlight, OLED pixels are **emissive** (organic materials emit light directly when forward biased by current), yielding infinite contrast ratio and low operating current.

---

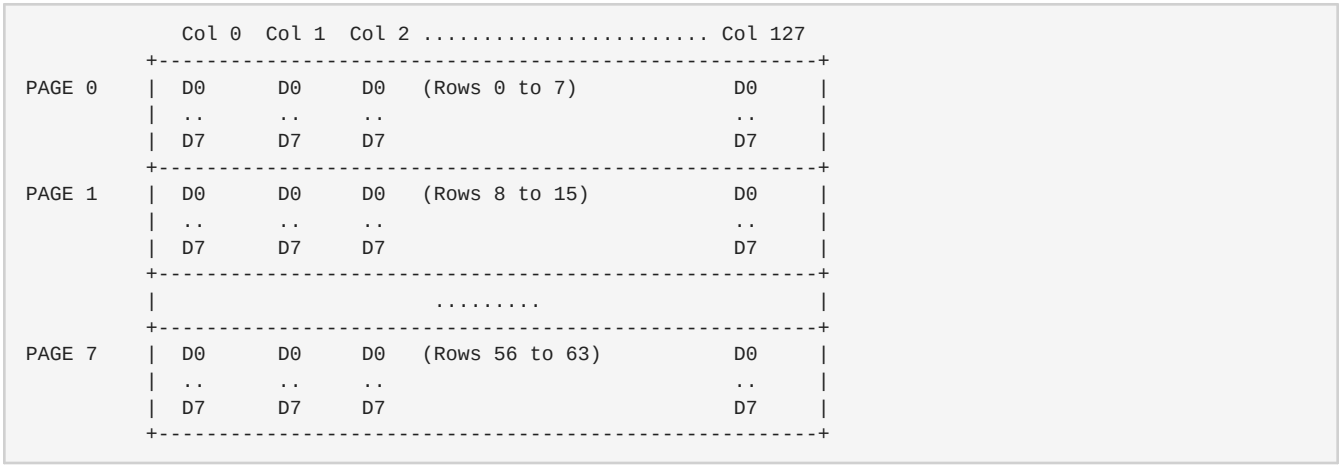
2. Technical Specifications & Memory Architecture

Parameter	Specification / Engineering Details
Display Resolution	128 columns $\times$ 64 rows = 8,192 dots (pixels)
Interface Protocol	Standard I2C (2-Wire: SCL, SDA)
I2C Device Address	7-bit: `0x3C` (STM32 HAL 8-bit shifted: `0x78` for Write, `0x79` for Read)
Operating Voltage (V_{DD})	3.3V Logic Supply
OLED Driving Voltage (V_{CC})	7.0V to 9.0V (Generated on-chip via Switched-Capacitor Charge Pump)
Current Consumption	All pixels OFF: ~1.2 mA; 50% pixels ON: ~15 mA; 100% pixels ON: ~28 mA
GDDRAM Size	128 bytes $\times$ 8 pages = **1,024 bytes (1 KB)**

---

3. GDDRAM (Graphic Display Data RAM) Mapping

The SSD1306 internal memory is organized vertically into **8 Pages (PAGE0 to PAGE7)**, each consisting of **128 columns**:



Key Memory Architectural Feature:

- Each byte written to GDDRAM represents a **vertical column slice of 8 vertical pixels**.
- **Bit 0** corresponds to the topmost pixel of the page; **Bit 7** corresponds to the bottom pixel.
- Because data cannot be written horizontally as standard scanlines, a single-pixel modification requires a read-modify-write operation or an MCU-side framebuffer.

---

4. Pure C Driver Architecture & SRAM Framebuffer

To eliminate third-party library memory overhead, a custom pure-C driver is implemented in [**`Core/Src/main.c`**](file:///c:/Users/GOPIKRISHNA/Downloads/MC_project_Pill_Box/MC_project_Pill_Box/Core/Src/main.c).

4.1 Local SRAM Framebuffer

```
static uint8_t SSD1306_Buffer[SCREEN_WIDTH * SCREEN_HEIGHT / 8]; // 128 * 64 / 8 = 1024 bytes
```

- **Memory Cost**: Consumes exactly **1,024 bytes (5%) of the STM32's 20 KB SRAM**, leaving 95% free for stack, heap, and peripherals.
- **Execution Flow**: All drawing primitives (**`DrawPixel`**, **`DrawLine`**, **`DrawRect`**, **`DrawString`**) manipulate bits in STM32 SRAM with sub-microsecond latency. The physical screen is refreshed only when **`ssd1306_UpdateScreen()`** transmits the full 1 KB frame over I2C in burst mode.

4.2 Pixel Addressing Bit-Math Formula

To toggle or color a pixel at cartesian coordinate (x, y) where $x \in [0, 127]$ and $y \in [0, 63]$:

$$\text{Byte Index} = x + \left(\left\lfloor\frac{y}{8}\right\rfloor \times 128\right)$$

$$\text{Bit Position within Byte} = y \pmod 8$$

$$\text{Bit Mask} = 1 \ll (y \pmod 8)$$

```
void ssd1306_DrawPixel(int x, int y, uint8_t color) {
    if (x < 0 || x >= SCREEN_WIDTH || y < 0 || y >= SCREEN_HEIGHT) return;
    if (color == 1) {
        SSD1306_Buffer[x + (y / 8) * SCREEN_WIDTH] |= (1 << (y % 8));   // Set pixel ON
    } else if (color == 0) {
        SSD1306_Buffer[x + (y / 8) * SCREEN_WIDTH] &= ~(1 << (y % 8));  // Clear pixel OFF
    } else {
        SSD1306_Buffer[x + (y / 8) * SCREEN_WIDTH] ^= (1 << (y % 8));   // Inverse / XOR
    }
}
```

4.3 Bresenham's Integer Line Drawing Algorithm

The driver uses Bresenham's incremental error line algorithm (**`ssd1306_DrawLine`**), avoiding floating-point computations:

$$\text{Error Increment } err = dx - dy$$

$$\text{Next Step}: \quad 2 \cdot err > -dy \implies x \leftarrow x + sx, \quad 2 \cdot err < dx \implies y \leftarrow y + sy$$

This executes purely using 32-bit integer arithmetic in Cortex-M3 hardware registers, drawing dividing borders in under $12\,\mu\text{s}$.

4.4 5x7 Monospace Font & Character Scaling

• The font table `font5x7[95][5]` stores 5 column bytes per ASCII character ($32\text{ to }126$).

• **Scaling Engine**:

• `size = 1`: Draws 6×8 pixels per character (normal menu, clock, and dates).

• `size = 2`: Scales each pixel into a 2×2 block, rendering crisp 12×16 characters used for high-urgency alarms (`"MEDICINE!"`, `"WRONG PILL!"`).

---

5. Electrical I2C Control Protocol & DC-DC Charge Pump

5.1 Control Byte Structure

In I2C communication with SSD1306, each transmission starts with a **Control Byte**:

• **Bit 7 (Co)**: Continuation bit (`0` = following bytes are data/commands only).

• **Bit 6 ($D/\bar{C}$)**: Data / Command select:

• `0x00`: Indicates following byte is a **Command** (e.g. contrast, page address).

• `0x40`: Indicates following bytes are **Display Data** to be written to GDDRAM.

5.2 Internal DC-DC Step-Up Charge Pump

OLED emission requires forward voltages between 7V and 9V, far exceeding the 3.3V supply. The SSD1306 integrates a switched-capacitor charge pump:

• Enabled during initialization using commands:

```
  ssd1306_WriteCommand(0x8D); // Charge pump control
  ssd1306_WriteCommand(0x14); // Enable charge pump (7.5V output)
```

• External flying capacitors connected across pins `C1P/C1N` and `C2P/C2N` on the breakout board alternately charge in parallel from 3.3V and discharge in series, stepping up the voltage with $>80\%$ efficiency.

---

6. Evaluator Oral Questions & Detailed Answers

Q1: Why is an MCU framebuffer necessary instead of writing directly to the OLED?

> ***Answer**:*
>
> *GDDRAM memory is mapped into 8-pixel vertical columns. If you want to change only 1 pixel at coordinate $(10, 15)$, you cannot write a single bit over I2C—you must write an entire 8-pixel vertical byte. If done without a framebuffer, the MCU would have to read the byte over I2C, modify the bit, and write it back. Doing this for every line, border, and character would flood the I2C bus with tens of thousands of read-modify-write transactions, causing severe screen lag. The 1024-byte SRAM buffer eliminates bus traffic during composition, and transfers the entire image in one 100 kHz burst.*

Q2: What addressing mode is configured in your SSD1306 driver?

> ***Answer**:*
>
> *We configure **Page Addressing Mode** (`0x20, 0x02` or manual page setting `0xB0 + page`). During screen update, the MCU loops through pages 0 to 7. For each page, it sets the column start address to 0 (`0x00, 0x10`) and streams 128 bytes*

of data directly.

Q3: What happens if the charge pump command (0x8D, 0x14) is omitted?

***Answer**:*

The SSD1306 logic core will operate and acknowledge all I2C commands, but the organic OLED pixels will remain completely dark. Without the charge pump step-up voltage, the potential across the pixel diodes cannot overcome the forward barrier potential ($V_f \approx 7\text{V}$), so zero current flows through the organic emissive layer.